ACKNOWLEDGEMENT

I express my sincere gratitude to Dr. Amit Kumar Dubey, Scientist/Engineer SF, LHD, CHSG, EPSA, ISRO SAC, for providing me the opportunity to work on a meaningful problem in Himalayan hydro-climatic data visualization and for his valuable guidance throughout the internship. His direction helped me understand both the scientific context of the work and the practical expectations of a research-support software platform.

I am equally thankful to Ms. Sachi Joshi, Assistant Professor, Department of Computer Engineering, DEPSTAR, CHARUSAT, for her academic guidance, encouragement, and continuous support during the preparation of this report. I also extend my thanks to Dr. Amit Nayak, Head of Department, Computer Science & Engineering, DEPSTAR, for his support and for providing an encouraging academic environment.

I gratefully acknowledge the support of ISRO SAC, Ahmedabad and DEPSTAR, CHARUSAT, for enabling this internship work. Finally, I thank my teachers, friends, and family for their motivation and support during the course of this internsh

ABSTRACT

This internship report presents the development of an interactive Web-based platform for visualization and preliminary analysis of Himalayan hydro-climatic and cryospheric datasets. The work was carried out at ISRO SAC with the objective of bridging the gap between raw scientific datasets and analysis-ready visualization for research use. The platform was designed to support historical, projected, and model-derived datasets such as ERA5-Land, CMIP6, SPHY outputs, MOD10A1 snow and albedo rasters, glacier boundaries, and user-uploaded NetCDF data.

The implemented system focuses on efficient data preparation, selective backend loading, map-based visualization, time-series generation, elevation-based filtering, subregion and glacier analysis, and support for long-term hotspot-style outcomes. The backend was developed using Python and FastAPI, while the frontend dashboard was implemented using React. A key contribution of the work is the conversion of large CSV outputs into optimized parquet files and the design of a lazy-loading query pipeline that improves responsiveness for large Himalayan datasets.

Overall, the internship contributed to the technical middle layer of a larger scientific workflow by creating a practical platform through which researchers can inspect spatial and temporal patterns before deeper hydrological interpretation. The platform is suitable for continued enhancement with advanced analytics, validation modules, and future scientific reporting workflows.

TABLE OF CONTENTS

LIST OF FIGURES

CHAPTER 1 INTRODUCTION

The Himalayan region plays a vital role in the hydrology of South Asia because it stores and regulates water through snow, glaciers, seasonal snowpack, soil moisture, and high-altitude catchments. These components influence river flow, downstream water availability, hydropower potential, agriculture, and ecosystem stability. Because of its complex terrain and high climate sensitivity, the region is frequently described as part of the Asian water towers.

Modern hydrological and cryospheric studies generate large volumes of spatial and temporal data from satellite observations, reanalysis products, climate projections, and physics-based model outputs. Although these datasets are scientifically valuable, they are often stored in formats such as CSV, NetCDF, GeoTIFF, shapefiles, or raster grids that are not convenient for quick scientific interpretation. Repeated manual inspection of large files slows down research-oriented exploration.

This internship work was positioned in the second phase of a broader project pipeline. The first phase focused on model generation and data preparation, while a future phase is expected to focus on deeper hydrological interpretation and scientific analysis. The internship contribution concentrated on extracting, organizing, visualizing, and making the datasets interactively usable through a WebApp.

The developed system provides a local and deployable interface in which users can select datasets, variables, year ranges, elevation bands, subregions, glaciers, and rectangular study areas. It visualizes map-based values, generates time-series plots, and supports long-term outcome views for hydro-climatic variables such as temperature, precipitation, snowfall, snow water equivalent, snow depth, wind speed, radiation, snow albedo, snow cover, and melt-related outputs.

1.1 BROAD AREA RELATED INTRODUCTION

The broad area of this internship lies at the intersection of the following domains:

- Himalayan hydrology and basin-scale water studies
- Cryosphere, snow, and glacier process analysis
- Hydro-climatic data visualization
- Remote sensing and reanalysis data processing

- Climate projection analysis
- Geospatial Web application development
- Scientific data engineering

Hydrological behavior in the Himalaya is strongly influenced by snowfall, snowmelt, glacier melt, rainfall-runoff interaction, and elevation-dependent temperature variation. For such studies, researchers rely on multiple datasets with different spatial resolutions, temporal frequencies, variable conventions, and file formats. A practical challenge therefore remains even after data generation: researchers still need a usable system to inspect what the data actually show.

The WebApp developed during this internship addresses that challenge by acting as a bridge between raw scientific products and future scientific analysis. It is designed to support map-based exploration, rapid comparison, and efficient access to filtered datasets without forcing scientists to repeatedly handle heavy scripts or GIS workflows.

1.2 OBJECTIVES

The primary objective of the internship was to develop an interactive WebApp for visualization and preliminary analysis of Himalayan hydro-climatic and cryospheric datasets. The specific objectives were as follows:

- Organize large hydro-climatic datasets into efficient formats suitable for local and Web-based analysis.
- Convert large CSV outputs into optimized parquet files for faster reading and filtering.
- Support multiple datasets including ERA5-Land, CMIP6, SPHY outputs, MOD10A1 rasters, glacier shapefiles, and uploaded NetCDF files.
- Develop a backend API capable of loading only the required dataset, year range, variable, date, elevation range, and region.
- Develop a frontend dashboard for interactive map-based visualization of Himalayan basin data.
- Allow users to inspect variables such as temperature, precipitation, snowfall, SWE, snow depth, wind speed, radiation, snow cover, snow albedo, and melt outputs.
- Implement elevation-based filtering for mountain hydrology analysis.
- Support subregion, glacier, and manually drawn rectangular region selection.
- Generate basin-wise and region-wise time-series plots.
- Implement long-term hotspot-style analysis for identifying areas with stronger temporal change.
- Prepare the WebApp for both online deployment and offline Windows usage.

1.3 MOTIVATION

The motivation for this work comes from a practical scientific need. Hydrological and cryospheric model outputs are often large, multi-year, and difficult to interpret directly. Scientists may need to repeatedly answer questions about snow storage, temperature variation with elevation, long-term regional change, precipitation behavior across selected years, or glacier-specific patterns. Without a dedicated visualization layer, each of these questions may require separate scripts and manual data handling.

The project is also motivated by performance and usability. Loading complete datasets into the browser is not practical, especially for large Himalayan domains. A system was therefore required in which the backend performs filtering and aggregation while the frontend receives only the data needed for the selected task. This design improves speed, stability, and scientific usability.

1.4 GOAL

The overall goal of the project is to create an analysis-ready visualization layer between physical model outputs and future hydrological interpretation. The broader workflow can be understood in three phases:

- Phase 1: Model and dataset generation, including hydrological simulations, satellite-derived products, and climate datasets.
- Phase 2: Data extraction, database preparation, and visualization through a responsive WebApp. This phase forms the main contribution of the internship.
- Phase 3: Scientific analysis and simulation interpretation using the prepared visualization platform for deeper research insights.

Accordingly, the goal of the internship was not to complete the full hydrological interpretation, but to build the technical platform that makes such analysis easier, faster, and more reliable for scientists and analysts.

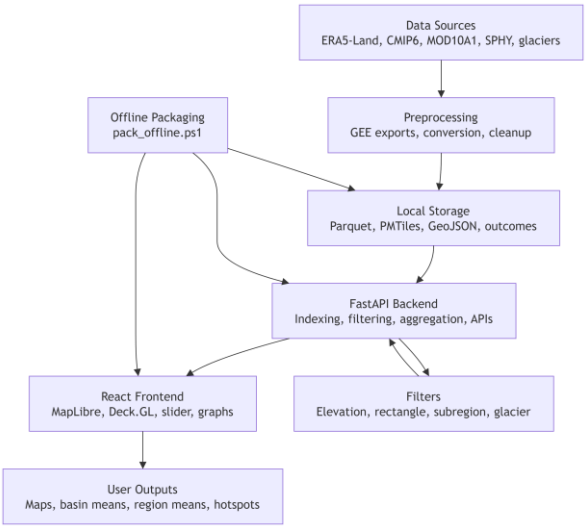

Fig. 1.1 Overall role of the internship work within the larger Himalayan analytics workflow.

CHAPTER 2 REVIEW OF RELATED LITERATURE AND EXISTING WORK

The Himalayan and Upper Indus regions have been studied extensively because they contain important snow and glacier reserves and support major downstream river systems. Existing literature highlights the importance of snow and glacier melt, precipitation variability, and elevation-dependent climate behavior in basin-scale hydrology.

Immerzeel et al. discussed how climate change can affect the Asian water towers and showed that meltwater contribution is especially important in the Indus basin [1]. Their work established the broader significance of cryosphere-dependent river systems and emphasized the need for tools that can support spatial interpretation of changing hydrological conditions.

Lutz et al. further showed that runoff in High Mountain Asia may increase in the short to medium term because of changing glacier melt and precipitation behavior [2]. This is directly relevant to the current project because it supports the need for interactive systems that can help researchers inspect where and how hydrological variables are changing over time.

Bolch et al. reviewed the state and fate of Himalayan glaciers and highlighted that glacier changes are spatially variable [3]. This observation motivates the inclusion of glacier-level and subregion-level filtering in the WebApp rather than relying only on basin-wide averages.

Distributed hydrological models such as SPHY provide a way to simulate rainfall-runoff, snowmelt, glacier melt, evapotranspiration, and related storage processes over complex terrain. Terink et al. presented SPHY as a spatially distributed model suitable for multiple hydrological applications [4]. Such models produce raster or gridded outputs that are best understood when they can be visualized spatially and interactively.

ERA5-Land has become a widely used reanalysis product for land-surface and hydro-climatic applications. Munoz-Sabater et al. described it as a state-of-the-art dataset for representing water and energy cycle conditions over land [5]. Likewise, CMIP6 and NEX-GDDP-CMIP6 provide valuable future climate projections for regional and local analysis

[6], [7]. Integrating these sources into a common platform can support historical and future comparisons.

MODIS snow products, especially MOD10A1, provide satellite-derived snow cover and albedo information that is useful for cryosphere studies [8], [9]. However, these products are often used through specialized GIS or scripting workflows. Existing scientific tools can process these datasets, but they typically require repeated manual handling. The contribution of the present work is to bring dataset access, map visualization, time-series plotting, region selection, glacier filtering, and analysis-ready outputs into one interactive Web-based system.

CHAPTER 3 METHODOLOGY

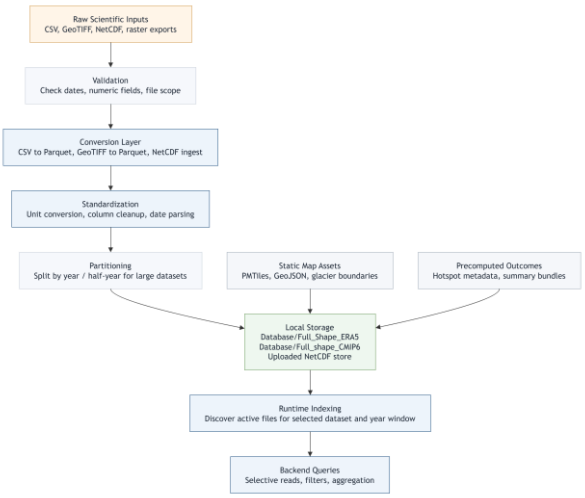

Fig. 3.1 data conversion and storage workflow.

The methodology adopted for this internship was designed to convert heterogeneous scientific datasets into a consistent exploratory environment without losing source-specific meaning. The core methodological idea was simple: keep each dataset scientifically recognizable, but make the interaction model common enough that users can move across data families through the same dashboard.

To achieve that, the work combined data preparation, storage optimization, backend query design, frontend visualization, and documentation. The methodology is therefore both computational and interpretive. It includes formal operations such as reprojection, unit conversion, temporal indexing, filtering, and aggregation. It also includes design decisions about what the user should see, what should be computed on demand, what should be precomputed once, and what assumptions should be made explicit in the interface and report.

3.1 DATA SOURCES

The WebApp was built to work with a layered collection of data sources rather than a single monolithic dataset. Historical hydro-climatic variables were provided through ERA5-Land derived point tables. Future scenario variables were provided through CMIP6-style outputs. Model-derived raster products were integrated from SPHY-

related workflows. Snow and albedo products were introduced through MOD10A1 GeoTIFF collections. Basin, subregion, and glacier vector assets were added for spatial context and selection. Later, the platform was extended with a NetCDF ingestion route so that users could bring additional scientific data into the same interface.

These source families differ not only in content but also in form. ERA5-Land and CMIP6 data were managed primarily as point-wise daily tables. SPHY-related data required GeoTIFF-to-parquet conversion to become dashboard-compatible. MOD10A1 retained a stronger raster character and was integrated with dataset-specific handling. Glaciers and basin boundaries were handled as vector geometries. NetCDF datasets required a conditional ingestion path because different files expose time and coordinate dimensions differently.

3.2 DATA ACQUISITION AND EARTH ENGINE PREPROCESSING

A major part of the upstream methodology was defined through Google Earth Engine export scripts. The supplied scripts sampled climate variables over the basin, aligned elevation information to the target dataset projection, and exported daily records with date, coordinate, elevation, and variable columns. This approach is important because it shifted the heavy spatial preprocessing out of the runtime application and into a reproducible preparation stage.

For ERA5-Land, the export logic selected variables such as temperature, snow water equivalent, snow depth, snowfall, total precipitation, solar radiation, and wind components. The script converted those into user-facing physical units, aligned the DEM to the ERA5-Land grid through bilinear resampling and reprojection, and sampled the basin at approximately 11 km scale. Longitude and latitude were added explicitly as bands so that the exported file preserved usable spatial coordinates.

For CMIP6, the export logic selected variables such as tas, tasmax, tasmin, pr, rsds, and sfcWind. The DEM was aligned to the CMIP6 grid and sampled at a coarser scale of approximately 25 km. Geometry columns were intentionally removed from the exported table after coordinate extraction so that the resulting files remained compact and directly usable for tabular analytics.

The methodological value of these Earth Engine scripts lies in grid-aware extraction. Instead of sampling elevation independently at a mismatched scale, the DEM was deliberately reprojected to the climate grid before export. This improves interpretive consistency because the elevation attached to a point corresponds more meaningfully to the same spatial support as the climate variable being visualized.

- Temperature in deg C = Temperature in K - 273.15
- Wind speed = sqrt(u_component^2 + v_component^2)
- Precipitation or snowfall in mm = source meters x 1000
- Solar radiation in MJ m^-2 day^-1 = source joules / 1,000,000 or source W m^-2 x 0.0864, depending on dataset definition

3.3 DATA CONVERSION AND UNIT HANDLING

Once the exported files were available locally, they were converted into runtime-oriented formats. The CSV-to-parquet conversion logic used chunked reading so that very large source files could be processed without requiring the whole file to be held in memory at once. During conversion, non-essential columns such as system:index and .geo were discarded, strict date parsing was applied, and numeric casting was performed only when values were consistently numeric.

This stage was not a cosmetic cleanup step. It was the first major performance optimization for the dashboard. CSV files are easy to inspect manually, but they are slow for repeated filtered analytics at scale. Parquet provides columnar storage, efficient compression, and better compatibility with selective reads. For a scientific dashboard that repeatedly filters by date, variable, elevation range, and region, this difference is substantial.

Unit handling was treated as part of the scientific contract of the application. The report, the code, and the documentation page all make variable naming explicit so that temperature_C means degrees Celsius, precipitation_mm means millimetres, temp_mean_C means projected mean temperature in degrees Celsius, and so on. Preserving this naming discipline reduces confusion when users switch between datasets that look similar but are derived differently.

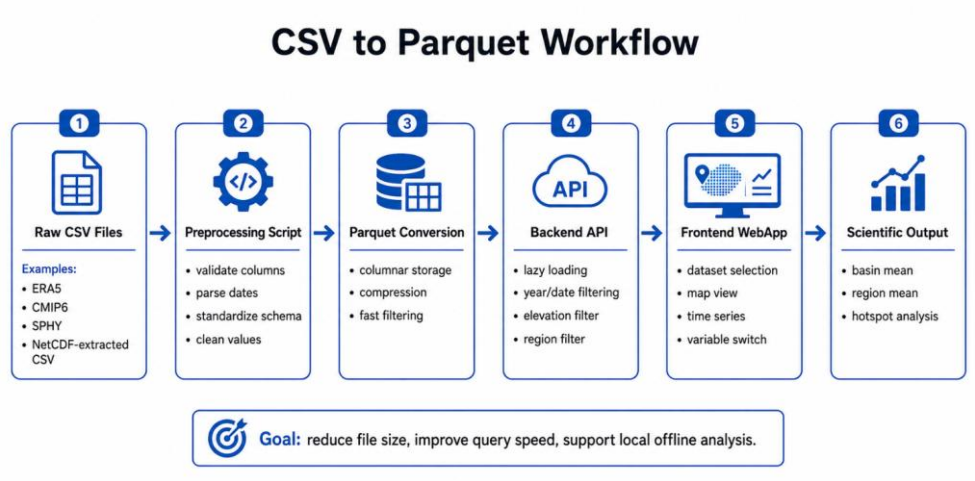

Fig. 3.2 data conversion and storage workflow.

3.4 STORAGE STRATEGY

The storage strategy evolved during the internship as dataset volume increased. The first principle was to keep runtime data in query-friendly structures, primarily parquet for table-like point datasets. The second principle was to separate source families by folder so that each ingestion path could remain understandable. The third principle was to reduce unnecessary loading by scoping files through the user-selected dataset and year range.

When some yearly files became too large for comfortable interactive use, the workflow was further optimized by dividing them into smaller temporal partitions such as six-month H1 and H2 files. This did not alter the scientific values. It only changed how the same data were organized on disk so that indexing and query loading could start faster and touch less data for narrower time windows.

Raster-based products were stored differently when needed. Some datasets were converted into parquet point layers because that matched the existing dashboard query logic. Others, such as MOD10A1-oriented products, retained raster-aware handling where that was more appropriate. Vector assets were stored in GeoJSON, shapefile-derived outputs, or PMTiles-ready map resources depending on their role.

3.5 OVERALL SYSTEM ARCHITECTURE

The system architecture follows a layered design in which storage, processing, interaction, and distribution are treated as connected but separable concerns. This separation was essential because the project changed repeatedly during development: new datasets were added, new map assets were introduced, a documentation module was created, hotspot analysis was implemented, and offline packaging became a deliverable. A tightly coupled ad hoc script would not have handled such evolution well.

At the storage layer, the system holds parquet datasets, GeoTIFF assets, vector boundaries, precomputed outcome files, uploaded NetCDF conversions, and packaged map resources. At the backend layer, FastAPI routes translate user requests into filtered data responses. At the frontend layer, React components coordinate controls, map rendering, plots, and documentation. At the packaging layer, an executable backend and built frontend are arranged for one-click offline use.

The architecture is local-first rather than cloud-first. That decision came from the original problem setting: the app was meant to run smoothly on a local machine with large scientific datasets. Performance decisions therefore favored reduced data transfer, bounded scopes, and selective local querying rather than permanent dependence on internet services.

3.6 BACKEND METHODOLOGY

The backend methodology centers on filtered query execution. Rather than preloading every dataset into one memory structure, the backend first determines the active dataset, the selected year window, and the relevant files for that scope. It then builds or reuses a lightweight index of dates, variables, and dataset statistics for that exact scope. Only after this scoping step does it answer the more detailed requests needed by the map and graph.

The backend exposes small, purpose-specific endpoints. Separate routes are used to list datasets, available years, dates, variables, and statistics. Data-serving endpoints return either map points for a selected day or aggregated time series for a basin or region. Additional routes handle subregions, glaciers, outcomes, hotspot trends, and

NetCDF upload. Keeping these responsibilities separate improved both debugging clarity and frontend coordination.

Query execution follows a deliberate filtering order. First, dataset and year-window filters establish the file scope. Second, date filters identify the relevant temporal slice. Third, elevation filters remove points outside the requested range. Fourth, spatial filters such as region bounds, subregion geometry, or glacier-linked selection are applied where appropriate. This order keeps later spatial operations smaller and therefore more responsive.

3.7 FRONTEND AND SPATIAL VISUALIZATION METHODOLOGY

The frontend methodology treats the map as the primary spatial canvas and the graph as the primary temporal summary. React manages application state, the dashboard controls drive data requests, Deck.GL renders data points efficiently, and MapLibre provides the map container and layer system. This combination makes it possible to update the map, legend, and graph together when a user changes filters.

Spatial visualization is based on stored longitude and latitude values rather than runtime interpolation. Each record is displayed at its recorded coordinate location. This is an important methodological point: the displayed point cloud is a direct view of sampled or prepared data positions, not a smoothed or re-gridded surface generated on the fly by the browser.

The map view uses a layer stack. A themed base layer provides general context. An India PMTiles vector layer provides country-scale administrative or boundary information. Basin and glacier overlays provide scientific context. Data points are then rendered on top using a variable-driven color scale. When a user selects a region, the corresponding rectangle or chosen geometry is retained visibly on the map so the graph context remains interpretable.

- Equal-interval legend thresholds: $t_k = \min + (k/5) \times (\max - \min)$ for $k = 1, 2, 3, 4$

3.8 RUNTIME LOADING AND PERFORMANCE STRATEGY

The platform was optimized around a simple performance principle: do not load more than the user is actively exploring. The home screen therefore asks the user to choose

both dataset and year range before the heavier dashboard is initialized. Later, the default year window was intentionally narrowed to avoid accidental full-range loading when users simply clicked through the initial screen.

The backend caches lightweight indexing information per dataset-year combination so that repeated scope changes do not always require full re-discovery. The frontend also helps performance by cancelling stale requests when rapid slider movement or repeated control changes occur. This reduces unnecessary backend work and prevents older responses from overwriting newer state.

Another performance decision was to precompute some long-window outputs once rather than derive them on every page load. Multi-decade band means for outcome pages are served from prepared parquet outputs. Similarly, splitting large files into smaller temporal partitions reduces the disk and parsing burden when only a subset is needed. These decisions were especially important because the app is local-first and must remain usable without depending on high-bandwidth server infrastructure.

3.9 REGION, GLACIER, AND GRAPH COMPUTATION LOGIC

The application supports several spatial analysis modes because a basin-wide view alone is often insufficient. Users can inspect the full basin, select a rectangle through a draw-like interaction, define a rectangle manually through latitude and longitude limits, choose a basin subregion from a dropdown, or focus on glacier-linked assets where available. These modes are different interface paths to the same methodological goal: narrowing the spatial support of the analysis while keeping the rest of the app behavior stable.

The basin mean graph computes an arithmetic mean across all currently valid points for each date within the selected time window. If x_i,d is the variable value at point i on day d and N_d is the number of valid points on that day, then the plotted basin mean is the average across those N_d values. The region mean graph follows the same arithmetic idea but restricts the set of points to the active rectangle or chosen geometry.

This computation is intentionally transparent. The graph is not an area-weighted climatology and not a modelled surface mean unless the source data themselves

already embody such modelling. It is the arithmetic mean of valid displayed-support points after the active dataset, year, elevation, and region filters are applied. This level of clarity matters because scientists may otherwise infer more statistical sophistication than the graph is actually claiming.

- Basin mean on date d: M_d = (1 / N_d) x sum(x_i,d)
- Region mean on date d: R_d = (1 / n_d) x sum(x_r,d) for points inside the selected region

3.10 LONG-TERM HOTSPOT ANALYSIS METHOD

The hotspot module was designed to show where a selected variable has changed most strongly over time across the basin. Methodologically, it does not compute a daily anomaly map. Instead, it looks across the full selected year window, summarizes each point annually, and fits a trend line through time for that point. The resulting slope represents the direction and magnitude of long-term change at that location.

To make the results readable, the app also computes distribution summaries of trend strength across points. Percentile thresholds such as P70, P85, and P95 are then used to separate stronger-change points from more moderate ones. A point above the P95 strength threshold is not being called extreme because of a universal physical rule; it is being classified as extreme relative to the distribution of point-wise trend strengths within the current filtered analysis set.

Minimum yearly coverage is an important control in this method. A trend is only meaningful when a point has enough valid annual observations across the selected window. Therefore, the hotspot computation can exclude points that do not meet the minimum annual availability threshold. This prevents unstable trend estimates from sparse or inconsistent temporal support.

- Point-wise annual trend model: Y = aX + b
- Where Y = annual mean value, X = year, a = trend slope, and b = intercept
- Trend strength classification is based on the absolute slope distribution, for example |a| compared against P70, P85, and P95

3.11 PRECOMPUTED OUTCOME GENERATION

Beyond on-demand hotspot calculation, the platform also introduced an outcomes section for precomputed long-term analytical views. One important example is the generation of mean maps over fixed multi-decade bands such as 1951-1975, 1976-2000, and 2001-2025 for ERA-based historical analysis. Instead of recalculating these summaries each time a user opens the outcome page, the mean values are prepared once through an external processing script and stored as parquet outputs.

The methodological advantage of this design is consistency and speed. Because the same precomputed files are served repeatedly, users do not experience long waits for the same broad summary product. At the same time, the computation remains auditable because the generation scripts are stored separately, making it clear how the output files were produced.

This separation between generation and serving is a recurring theme in the project. Heavy transformations are performed once where possible; interactive inspection is reserved for the dashboard runtime. That division is especially helpful when the same scientific outcome is meant to be viewed multiple times by different users.

3.12 NETCDF AND RASTER INGESTION EXTENSIONS

Later in the internship, the platform was extended beyond the original ERA5-Land and CMIP6 point-table pattern. One extension introduced NetCDF upload. The idea was to let a user provide an additional gridded scientific file, allow the backend to inspect its coordinate structure, convert it into an app-compatible dataset, and expose it in the same dataset selection workflow. Because NetCDF files vary in structure, the ingestion path was kept intentionally modular and separate from the main historical dataset code path.

A second extension involved GeoTIFF-based scientific data. SPHY-related melt and model outputs were converted into parquet through a dedicated GeoTIFF-to-parquet conversion step so that they could use much of the existing dashboard logic. MOD10A1-oriented data represented another raster integration path where satellite-like snow and albedo information had to be incorporated without harming the existing dataset modules.

Methodologically, these extensions demonstrate that the platform is format-aware rather than format-blind. New data are not forced into the system carelessly. Instead, each data family is given an ingestion path that respects its structure, then connected to the shared dashboard contract only after that translation is defined.

3.13 VALIDATION, REPRODUCIBILITY, AND LIMITATIONS

Validation in this project has two layers. The first layer is structural validation: checking that coordinates, dates, variable names, year ranges, and units are handled consistently, and that application outputs respond logically to filter changes. The second layer is scientific cross-checking: comparing representative values or spatial patterns against source expectations or known dataset behavior. For example, point-based comparisons against Earth Engine-derived reference logic can help confirm that basin summaries or long-term means are within a reasonable interpretation range.

Reproducibility is supported through explicit file organization, documented preprocessing scripts, named variables with units, and user-visible filters. A scientist can record dataset, year range, variable, elevation band, region choice, and module type when describing any result. This does not make the system equivalent to a full provenance tracker, but it does greatly improve repeatability over manual ad hoc plotting workflows.

Several limitations remain and are intentionally documented. Basin and region means are arithmetic means rather than area-weighted means. Color scales on the dynamic map are relative to the current filtered min-max range rather than fixed climatological thresholds. Cross-dataset comparisons require caution because ERA5-Land, CMIP6, MOD10A1, and model outputs differ in spatial support and scientific meaning. Finally, the application is an exploratory interface, not a substitute for full domain validation, uncertainty analysis, or publication-grade statistical inference

CHAPTER 4 IMPLEMENTATION

The implementation phase transformed the methodological ideas of the previous chapter into a working scientific application. The WebApp can be understood as a collection of coordinated modules rather than a single screen: data storage, backend services, frontend interaction, spatial assets, analytical extensions, documentation, and offline packaging. During the internship, each of these modules evolved through repeated testing, debugging, and extension as new requirements were introduced.

A key implementation challenge was maintaining stability while the scope expanded. The platform began as a local dataset viewer for a narrower basin-focused use case, but later incorporated broader basin geometry, more datasets, new raster families, glacier overlays, region-selection improvements, hotspot analysis, long-term outcomes, documentation pages, and portable offline packaging. The code therefore had to remain extensible while continuing to support earlier functionality.

4.1 DATA LAYER

The data layer of the final application is organized around distinct asset families so that each can be handled by the correct runtime logic. Historical ERA5-Land point tables, future CMIP6 point tables, SPHY-related parquet outputs, MOD10A1 GeoTIFF folders, uploaded NetCDF derivatives, basin and glacier vectors, and map assets are stored separately rather than mixed into one generic directory. This organization improves maintainability and also reflects the fact that these files are scientifically different.

At implementation level, this separation made it easier to add new modules without destabilizing old ones. For example, glacier overlays could be introduced without changing how ERA5-Land values were queried. Similarly, MOD10A1 data could be connected through a dedicated raster-aware path instead of forcing the backend to pretend that every dataset behaves identically.

4.2 BACKEND IMPLEMENTATION

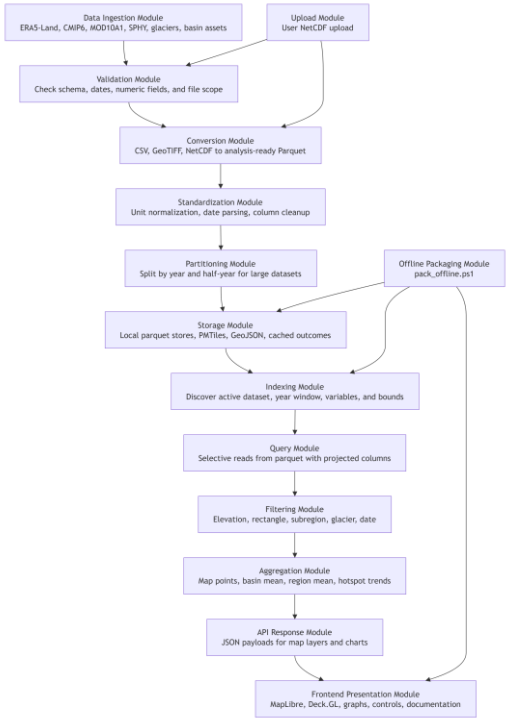

Fig. 4.1 System architecture of the implemented Himalayan analytics WebApp.

The backend was implemented around a FastAPI application that exposes the minimal set of routes necessary for the dashboard to remain responsive and scientifically understandable. Startup logic initializes dataset state. Dataset discovery routes return available datasets and years. Scope-building routes prepare dates, variables, and statistics. Analytical routes then serve one-date point maps, basin means, region means, hotspot trends, subregion geometry, glacier search results, and outcome products.

The implementation emphasized selective IO. Instead of reading every possible column from every relevant file, query paths attempt to read only the columns required for the active request. This matters when working with parquet because column projection can reduce memory movement significantly. The same idea applies to temporal scoping: only the selected year range is indexed and queried.

The backend also had to evolve to support new ingestion types. NetCDF upload introduced a separate conversion module rather than embedding fragile format-detection logic into the main query file. GeoTIFF conversion scripts for SPHY-like outputs were similarly kept

separate so the core dashboard code could remain focused on serving structured data rather than handling every raw format directly.

4.3 FRONTEND IMPLEMENTATION

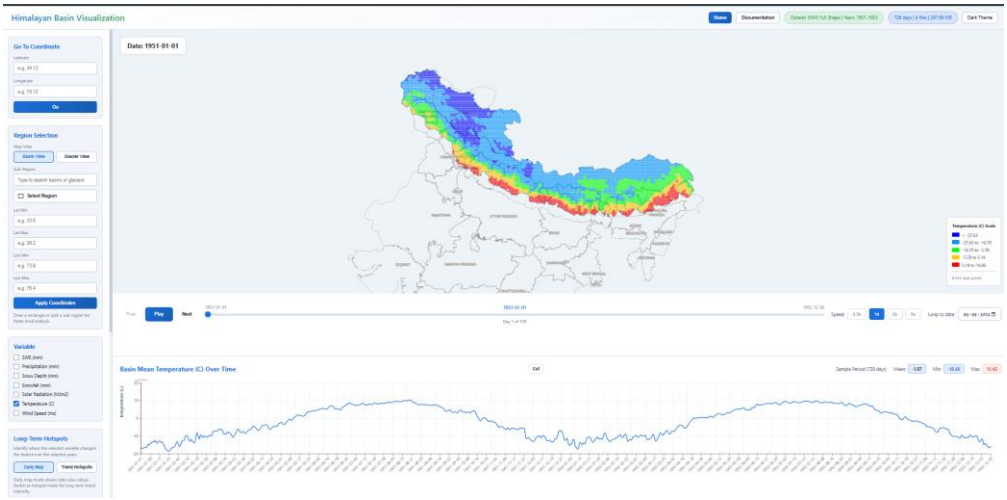

Fig. 4.2 Main dashboard showing map-based visualization.

The frontend was built in React and organized around two major user stages: the startup form and the analytical dashboard. The startup form asks the user to choose a dataset and year range before entering the heavier interactive view. This was not merely a user-interface choice; it was a performance control that prevents accidental loading of very large historical windows.

Once the dashboard opens, the frontend coordinates multiple stateful controls: dataset, year window, variable, date, elevation range, theme, region-selection mode, glacier or subregion choice, graph zoom state, hotspot parameters, and outcome page selections. The implementation challenge was keeping these controls synchronized so that changes in one place update the correct map and graph modules without creating contradictory state.

Special attention was given to the visibility of scientific context. The frontend shows legends, region selection state, point counts, dataset labels, and documentation content so that the user is not left guessing what is being displayed. The documentation page itself became an implementation component, not a separate external note, because the scientific meaning of the interface depends on that explanatory layer.

4.4 MAP HANDLING AND SPATIAL LAYERS

Map handling underwent several iterations during development. The final map composition uses a multi-layer approach in which a base map provides general orientation, a PMTiles-based India layer adds boundary context, the active basin boundary is overlaid clearly, and dataset points are rendered above it. Additional overlays such as glaciers, selected subregions, or user-drawn rectangles are added as needed.

This arrangement was chosen to solve a practical visualization problem. If boundaries are too bold, the data colors become hard to read. If boundaries are too faint, the basin context is lost. The map styling was therefore adjusted so that basin and country boundaries remain thin enough not to dominate the data, yet visible enough to orient scientific interpretation. Theme toggling was also extended to work with the map so that the dashboard and spatial scene stay visually consistent.

Another important implementation lesson was that offline map assets are different from online basemap tiles. The PMTiles integration required its own path handling and zoom behavior checks. Issues such as labels disappearing or boundaries only appearing at certain zoom levels had to be debugged by examining both frontend styling and the PMTiles asset behavior.

4.5 REGION, SUBREGION, AND GLACIER SELECTION

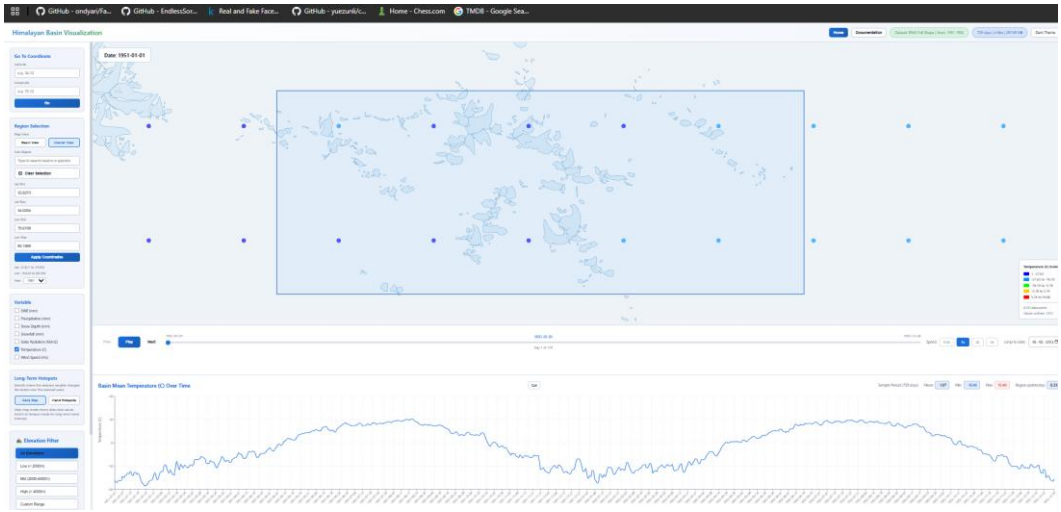

Fig. 4.3 Region and glacier selection.

Region selection became one of the most important analytical features of the application. The initial requirement was rectangle-based spatial selection so that users could isolate a local study area and generate a time-series graph for that region. Over time, the feature was extended so that the selected rectangle remained visible until cleared, point counts inside the selected region were shown near the graph, and the selected area could also be specified numerically through latitude-longitude bounds.

A separate but related feature was subregion and glacier selection. Basin subregions and glacier names can be searched or chosen from interface controls. One important design correction made during development was to ensure that choosing a subregion from the menu does not automatically behave like a rectangle selection. That separation matters because a user may want to choose a named subregion for context while still drawing a smaller rectangle inside it for focused analysis.

The glacier integration extended the same idea further. Glacier shapefiles were loaded as visible map layers and their names were incorporated into the region-related interface. This created a more domain-aware selection system in which the user can move between freehand geographic focus and named scientific entities.

4.6 TIME NAVIGATION AND GRAPH INTERACTION

The dashboard was designed to support both spatial browsing and temporal browsing. Play and pause controls let the user step through the available dates in sequence, while a date slider and date selection tools support direct jumps. This transforms the map into a time-aware scene rather than a static figure.

Graph interaction was also extended beyond a simple always-full-range view. A cut or zoom mode allows the user to define a shorter graph interval by clicking a start date and an end date on the plot. Once selected, the reduced range remains active even when the variable changes, which makes it easier to compare different variables over the same event window or study period.

This part of the implementation is important because scientific insight often emerges from focusing on a narrower period rather than always looking at the full range. Preserving the selected graph window across variable changes was therefore a practical but meaningful usability improvement.

4.7 HOTSPOT AND OUTCOME MODULES

The hotspot module and the outcomes module represent the shift from basic exploration to higher-level analytical products. The hotspot module computes long-term trend strength across points for a selected variable and returns both a map and summary metrics such as points analyzed, strong-hotspot counts, mean strength, maximum strength, and percentile thresholds. These summaries are useful because they translate a large field of local trend estimates into a compact descriptive overview.

The outcomes module goes a step further by loading precomputed long-term band means for fixed historical windows. This reduces compute time at runtime and creates a dedicated interface for long-term mean maps rather than only instantaneous daily fields. The left-side controls on the outcome page let the user choose a variable and a multi-year band, after which the spatial mean map for that prepared output is displayed.

From an implementation perspective, these modules demonstrate an important architectural principle of the project: not all scientific views need to be computed the same way. Some should remain on-demand and filter-responsive, while others are better prepared once and served repeatedly.

4.8 SPHY, MOD10A1, AND GEOTIFF INTEGRATION

A major later-stage implementation task was to connect raster-driven scientific products to the same application without harming existing modules. For SPHY-related outputs, the chosen solution was to create a dedicated GeoTIFF-to-parquet conversion script. This allowed the resulting data to use much of the same dashboard machinery as the original point-table datasets while preserving their spatial meaning.

For MOD10A1 monthly GeoTIFF data, the integration challenge was different. These products already carry a raster identity that is central to their scientific interpretation. Therefore, the implementation had to respect that dataset family rather than force it through an inappropriate generic path. The app was extended so that MOD10A1 could be selected as its own dataset group and displayed using logic appropriate to its raster-derived structure.

These integrations show that the platform architecture is not locked to a single source format. Instead, it can grow by attaching new translators and new dataset modules while keeping the main analytical workflow stable.

4.9 NETCDF UPLOAD MODULE

The NetCDF upload feature was introduced to make the platform more open-ended. Rather than limiting users to the datasets already bundled with the app, the system can accept a user-supplied NetCDF file, inspect its structure, convert it into parquet or a similarly compatible internal representation, and register it as an additional dataset. The design requirement here was strong isolation: the new ingestion path should not break the existing historical data logic.

Because NetCDF files vary greatly, the upload interface also required user guidance. The implementation therefore included a help or information control describing the general characteristics that make a file easier to ingest, such as a usable time coordinate and recognizable spatial dimensions. At the same time, the backend logic was extended so that it could be more tolerant of realistic file variation rather than demanding one rigid schema for every case.

This feature is valuable for research because many teams receive intermediate data products from collaborators in NetCDF form. The upload module lets such products be brought into the same exploration environment instead of forcing a separate toolchain for every new file.

4.10 DOCUMENTATION MODULE AND SCIENTIFIC COMMUNICATION

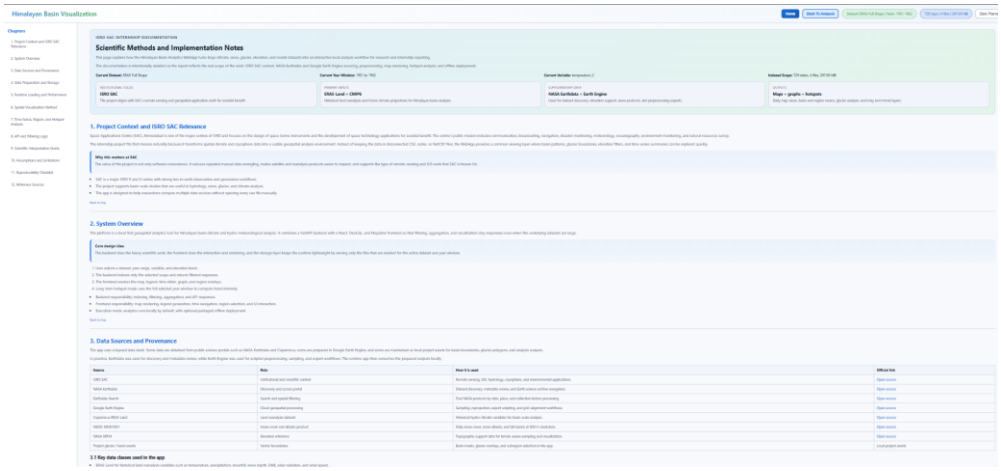

Fig. 4.4 Documentation module

One of the most distinctive implementation additions was the documentation page inside the frontend itself. This page explains project context, ISRO SAC relevance, source provenance, preprocessing logic, map behavior, API design, analytical interpretation, limitations, and reproducibility considerations. It effectively acts as the scientific user manual of the WebApp.

This module matters because scientific software can otherwise become opaque very quickly. A user might see a map and a graph but remain unsure about units, source resolution, what the legend means, or how a hotspot is computed. By embedding documentation directly into the app, the project reduces that uncertainty and improves responsible use.

For the internship report, this documentation module also served as an internal source of well-structured methodological explanation. The presence of that module shows that the internship deliverable was not only code but also interpretive support for future scientific users.

4.11 OFFLINE PACKAGING AND DEPLOYMENT READINESS

A practical deployment requirement of the project was that the application should be usable by colleagues who do not have Python, Node.js, or development tooling preinstalled. To satisfy that requirement, the platform was packaged into an offline

Windows bundle containing a compiled backend executable, the built frontend files, runtime assets, map resources, and a one-click launcher script.

This packaging step required the backend to serve the built frontend and local assets in release mode so that a user could start the application without manually starting separate development servers. It also required attention to folder layout, map assets, and portability of the runtime environment.

The offline bundle is not only a convenience feature. It is part of the broader design philosophy of the internship: the value of a scientific interface increases significantly when non-developer collaborators can run it reliably on their own machine.

4.12 CHALLENGES, DESIGN DECISIONS, AND LESSONS LEARNED

The development process exposed several recurring challenges. The first was scale: even when datasets are locally available, they may be too large for naive interactive handling. This challenge was addressed through parquet conversion, temporal partitioning, selective year-window loading, and backend-side aggregation.

The second challenge was heterogeneity. ERA5-Land, CMIP6, SPHY outputs, MOD10A1 rasters, glacier vectors, and uploaded NetCDF files do not naturally behave the same way. The design decision here was not to collapse them into one careless abstraction, but to create a shared interaction framework with dataset-specific ingestion and serving logic underneath.

The third challenge was cartographic clarity. Map layers that are too heavy or too numerous can obscure scientific values rather than reveal them. Several iterations were therefore spent on line thickness, color balance, PMTiles visibility, theme support, and layer ordering so that data points remain visually readable.

The fourth challenge was scientific communication. Features such as hotspot percentiles, mean graphs, region selection, or dynamic legends can be misunderstood if their logic is not explained. This led directly to the addition of the documentation module and to the emphasis in this report on formulas, source provenance, and interpretive notes.

A final lesson is that scientific app development is a form of methodological work. Every engineering choice changes what the user can see quickly, compare fairly, or reproduce later. In that sense, software architecture and scientific transparency are not separate concerns in this project; they are part of the same design responsibility.

CHAPTER 5 CONCLUSION

This internship resulted in the development of a substantial and evolving scientific WebApp for Himalayan hydro-climatic and cryosphere-oriented data exploration. The contribution is best understood not as a single algorithm but as an integrated platform that turns raw and heterogeneous scientific files into an interactive analytical workflow. Through this work, map layers, graphs, region filters, glacier overlays, hotspot analysis, outcome pages, and documentation were brought into one coherent environment.

The project sits in the middle phase of a broader scientific pipeline. Upstream data generation and model preparation provide the raw inputs. Downstream hydrological interpretation and simulation can build upon the interface created here. The internship therefore added value by improving the transition from data possession to data understanding, which is often where research productivity is lost.

The platform also demonstrates that local-first scientific tooling can remain sophisticated. It supports multiple dataset families, flexible spatial filters, long-term analysis modules, and offline distribution for non-developer users. At the same time, the report makes clear that the app is an exploratory and preparatory tool rather than a replacement for full domain validation or advanced modelling.

The internship contribution can be summarized across several connected layers. At the data-engineering layer, the work established usable preprocessing paths from exported CSVs and GeoTIFF files to runtime-friendly parquet structures. At the backend layer, it created a selective query architecture capable of serving maps, graphs, hotspot results, and long-term outcomes without full-range loading by default.

At the frontend layer, the work produced a practical scientific dashboard with synchronized spatial and temporal controls, theme-aware mapping, glacier and subregion selection, manual region definition, graph zooming, and map-based interpretation support. At the communication layer, the work added in-app documentation and a detailed report so that scientists can understand what the application is actually computing and displaying. Finally, at the usability and deployment layer, the project delivered an offline bundle pathway so that collaborators without technical setup can still run the application. This is a

meaningful contribution because research software only creates real value when it can be used by the intended audience.

REFERENCES

[1] W. W. Immerzeel, L. P. H. van Beek, and M. F. P. Bierkens, "Climate Change Will Affect the Asian Water Towers," Science, Vol. 328, No. 5984, pp. 1382-1385, 2010, American Association for the Advancement of Science.

[2] A. F. Lutz, W. W. Immerzeel, A. B. Shrestha, and M. F. P. Bierkens, "Consistent Increase in High Asia's Runoff Due to Increasing Glacier Melt and Precipitation," Nature Climate Change, Vol. 4, No. 7, July 2014, Nature Publishing Group.

[3] T. Bolch, A. Kulkarni, A. Kaab, C. Huggel, F. Paul, J. G. Cogley, H. Frey, J. S. Kargel, K. Fujita, M. Scheel, S. Bajracharya, and M. Stoffel, "The State and Fate of Himalayan Glaciers," Science, Vol. 336, No. 6079, 2012, American Association for the Advancement of Science.

[4] W. Terink, A. F. Lutz, G. W. H. Simons, W. W. Immerzeel, and P. Droogers, "SPHY v2.0: Spatial Processes in HYdrology," Geoscientific Model Development, Vol. 8, 2015, Copernicus Publications.

[5] J. Munoz-Sabater et al., "ERA5-Land: A State-of-the-Art Global Reanalysis Dataset for Land Applications," Earth System Science Data, Vol. 13, 2021, Copernicus Publications.

[6] B. Thrasher et al., "NASA Global Daily Downscaled Projections, CMIP6," Scientific Data, 2022, Nature Portfolio.

[7] NASA Earth Exchange, "NEX-GDDP-CMIP6: NASA Earth Exchange Global Daily Downscaled Projections," Dataset Documentation, NASA.

[8] National Snow and Ice Data Center, "MODIS/Terra Snow Cover Daily L3 Global 500 m SIN Grid, Version 61," MOD10A1 Dataset Documentation, NASA NSIDC DAAC.

[9] Google Earth Engine Data Catalog, "MOD10A1.061 Terra Snow Cover Daily Global 500 m," Dataset Documentation, Google.

[10] Copernicus Climate Data Store, "ERA5-Land Hourly Data from 1950 to Present," Dataset Documentation, ECMWF and Copernicus Climate Change Service.